

Week 2 - Day 6a Supplementary: Dictionaries, Functions

Run the following cell before doing the exercises.

```
# Run this function
def check(fn, *input, expected):
    result = fn(*input)
    if expected != result:
        print(
            f"Function should return the value {expected}, it is "
            f"returning the value {result}."
        )
    else:
        print(f"Congratulations, the test case passed!")
```

Exercise 1: Is Even (without if statements!) []

1.1 Write a function `isEven` that gets a number and returns `True` if the number is even, or `False` otherwise. Your function **cannot** use `if` statements.

```
#TODO: Write isEven function without if statements
```

Run the following code cell to check your solution.

```
check(isEven, 0, expected=True)
check(isEven, 20, expected=True)
check(isEven, -10, expected=True)
check(isEven, 5, expected=False)
check(isEven, 1, expected=False)
```

1.2 Write a function `printIsEven` that gets a number `n` and prints "`n is even`" if the number is even or "`n is odd`" otherwise. Your function **cannot** use `if` statements.

```
#TODO: Write printIsEven function without if statements
```

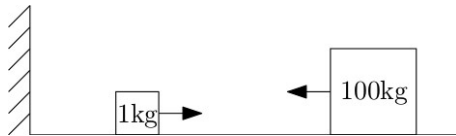
Run the following code cell to check your solution.

```
printIsEven(2) ## Expected: n is even
printIsEven(1) ## Expected: n is odd
printIsEven(3) ## Expected: n is odd
printIsEven(9) ## Expected: n is odd
printIsEven(0) ## Expected: n is even
printIsEven(-2) ## Expected: n is even
```

```
printIsEven(-1) ## Expected: n is odd
printIsEven(24) ## Expected: n is even
```

Exercise 2: Collisions! []

Suppose that you have two boxes B_1 and B_2 on the floor and they have different weights m_1 and m_2 , respectively. B_1 is on the left and B_2 is on the right. $m_2 \geq m_1$. There is also a wall on the left of B_1 . Now B_2 box is moving toward B_1 box with initial velocity v_2 , and B_1 has velocity $v_1=0$ (not moving). Ignore all the frictions and extra cost of energy. **Question:** count the total number of collisions between the B_1 and B_2 and between B_1 and the wall.



When B_1 hit the wall, the new velocity of B_1 becomes $-v_1$ (when v_1 is the velocity before the collision with the wall).

For the case that B_1 and B_2 collide, the followings are two useful laws in physics:

Let v_1, v_2 be the velocity of the boxes before a collision, and let $\hat{v}_1$ and $\hat{v}_2$ be the velocity after the collision.

The Law of Conservation of Kinetic Energy:

$$\frac{1}{2}m_1 v_1^2 + \frac{1}{2}m_2 v_2^2 = \frac{1}{2}m_1 \hat{v}_1^2 + \frac{1}{2}m_2 \hat{v}_2^2$$

The Law of Conservation of Momentum:

$$m_1 v_1 + m_2 v_2 = m_1 \hat{v}_1 + m_2 \hat{v}_2$$

Let's learn how to use Python functions to simulate it.

The following functions compute the new velocities after a collision based on the equations above.

```
def boxhit(m1, m2, v1, v2):
    new_v1 = ((m1 - m2) * v1 + 2 * m2 * v2) / (m1 + m2)
    new_v2 = ((m2 - m1) * v2 + 2 * m1 * v1) / (m1 + m2)
    return new_v1, new_v2

def wallhit(v1, v2):
    v1 = -v1
    return v1, v2
```

Fill in the following code cell to run the simulation

```

#TODO: Finish simulate function

# moving leftward means the velocity is positive and moving rightward
means that the velocity is negative
# When v1 is smaller than 0 or equal to 0, B1 box must move rightward
(or it's not moving) and there would be a boxhit.
# When v1 is greater than 0, B1 box is moving leftward and there would
be a wallhit.
def simulate(m1, m2, v1 , v2):
    collision_num = 0
    while True:
        # code here

        if abs(v2) >= abs(v1) and v2 < 0:
            break
    return collision_num

```

Run the following and see if you can find the pattern of the number of collisions when m_2 is a multiple of 100.

```

print(simulate(1, 10 ** 0, 1, 1000))
print(simulate(1, 10 ** 2, 1, 1000))
print(simulate(1, 10 ** 4, 1, 1000))
print(simulate(1, 10 ** 6, 1, 1000))
print(simulate(1, 10 ** 8, 1, 1000))

check(simulate, 1, 10 ** 0, 1, 1000, expected=4)
check(simulate, 1, 10 ** 2, 1, 1000, expected=32)
check(simulate, 1, 10 ** 4, 1, 1000, expected=314)
check(simulate, 1, 10 ** 6, 1, 1000, expected=3142)
check(simulate, 1, 10 ** 8, 1, 1000, expected=31416)
print("The result is really interesting, and when m2 is 100 ** n, the
number of collisions divided by 10 ** n converges to pi!")

```

Exercise 3: Grading []

Dr. Kang is assigning grades to his students. Help him write a grading function that takes a student's score (out of 100) as input and returns a letter grade. The cutoffs for grades are included below.

Range	Grade
[95-100]	A+
[85-94]	A
[80-84]	A-
[75-79]	B+
[70-74]	B
[65-69]	B-

Range	Grade
[60-64]	C+
[55-59]	C
[50-54]	C-
[45-49]	D
[0-45]	F

3.1 Define a helper function named `isValidScore` that takes in a score as input and returns a boolean. If the score that is passed into this function is between 0 and 100 (both inclusive) return True otherwise return False.

#TODO: Define helper function

```
def isValidScore(s):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.
check(isValidScore, 50, expected=True)
check(isValidScore, 101, expected=False)
check(isValidScore, -101, expected=False)
check(isValidScore, 100, expected=True)
check(isValidScore, 0, expected=True)
```

3.2 Define a function named `grader` that has one parameter. You can name the parameter score. Within the function you should call the `isValidScore` function you defined in 1.1 by passing in the score. If the call to `isValidScore` returns True your function must return the string `Valid Score`. Otherwise it should return the string `Invalid Score`.

#TODO: Write grader function

```
def grader(score):
    # Put your code below this comment
    return False

# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.
check(grader, 50, expected='Valid Score')
check(grader, 150, expected='Invalid Score')
check(grader, -10, expected='Invalid Score')
```

3.3 Modify the `grader` function you defined in 1.2 so that it returns the letter grade for the score. Use the grade cutoffs written above. If the score is greater than or equal to 95 your

function should return an A+, if the score less than 95 and greater than or equal to 85 your function should return an A and so on. You should still use the `isValidScore` to check for invalid input and it must return the string `Invalid Score` for any invalid scores.

```
#TODO: Mofify grader function

def grader(score):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
first code
# cell in this notebook.

check(grader, 90, expected='A')
check(grader, 56, expected='C')
check(grader, 70, expected='B')
check(grader, -50, expected='Invalid Score')
check(grader, 105, expected='Invalid Score')
check(grader, -10, expected='Invalid Score')
```

Exercise 4: Reverse Number []

Define a function `reverseNumber` which takes in a postive integer variable `x` and return a reversed version of the number.

Example: `reverseNumber(12345) = 54321`

4.1 Try to use string to solve it.

```
#TODO: Write reverseNumber with string

def reverseNumber1(x):
    pass #delete this line and add your code here
```

4.2 Try not to use string to solve it.

```
#TODO: Write reverseNumber2 without string

def reverseNumber2(x):
    pass #delete this line and add your code here
```

Use the following code cell to test your solution

```
check(reverseNumber1, 12345, expected=54321)
check(reverseNumber1, 1245, expected=5421)
check(reverseNumber1, 2345, expected=5432)
check(reverseNumber2, 12345, expected=54321)
```

```
check(reverseNumber2, 1245, expected=5421)
check(reverseNumber2, 2345, expected=5432)
```

Exercise 5: Ancient Ethiopian Multiplication []

Ethiopians have an ancient method of multiplying integers using only addition, doubling, and halving. Here is how the method works:

1. Take two numbers to be multiplied and write them down at the top of two columns.
1. In the left-hand column repeatedly halve the last number, discarding any remainders, and write the quotient below the last in the same column, until you write a value of 1.
2. In the right-hand column repeatedly double the last number and write the result below. Stop when you add a result in the same row as where the left hand column shows 1.
3. Examine the table produced and discard any row where the value in the left column is even.
 - a. Sum the values in the right-hand column that remain to produce the result of multiplying the original two numbers together

For example: 17×34 Halving the first column:

17	34
8	
4	
2	
1	

Doubling the second column:

17	34
8	68
4	136
2	272
1	544

Strike-out rows whose first cell is even:

17	34
8	68
4	136
2	272
1	544

Sum the remaining numbers in the right-hand column:

17	34
8	--
4	---
2	---
1	544
Total	578

So 17 multiplied by 34, by the Ethiopian method is 578.

5.1 Define a helper function named `isEven` that takes in an integer and returns `True` if the number is even or `False` if it's odd.

```
#TODO: Write isEven
def isEven(n):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.

check(isEven, 2, expected=True)
check(isEven, 1, expected=False)
check(isEven, 15, expected=False)
```

5.2 Define another helper function named `halveIt` that takes in an integer and then returns a list with the halved values till it reaches 1. Make sure the list you are returning is sorted from largest to smallest. For example, if you give it the number `17` it should return the list `[17, 8, 4, 2, 1]`, and if you give it the number `105` it should return the list `[105, 52, 26, 13, 6, 3, 1]`.

```
#TODO: Write halveIt function
def halveIt(n):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.

check(halveIt, 17, expected=[17, 8, 4, 2, 1])
check(halveIt, 105, expected=[105, 52, 26, 13, 6, 3, 1])
```

5.3 Define another helper function named `doubleIt` that takes in two argument. The first argument should be the number that is going to be doubled, and the second should number

should be the target length of the list to be returned. (If the second number is 5, you will double the starting number 4 times.) Your function should return a list with the doubled values. Make sure the list is sorted from smallest to largest. For example, if you pass **34** and **5** to your function it should return the list **[34, 68, 136, 272, 544]**.

```
#TODO: Write doubleIt function
def doubleIt(num, n):
    # Put your code below this line
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.

check(doubleIt, 34, 5, expected=[34, 68, 136, 272, 544])
```

5.4 Define a helper function named **sumLists** that takes in two lists. The two lists must have equal length. You should loop through each corresponding pair of elements in the two lists, and if the element in the first list is not even, then you should add the corresponding element from the second list to your total. Your function should use the **isEven** function you defined earlier to check if the element in the first list is even or not. For example, calling **sumLists([1, 2, 4, 8, 17], [34, 68, 136, 272, 544])** should return 578.

```
#TODO: Write sumLists function
def sumLists(list1, list2):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
# first code
# cell in this notebook.

check(sumLists, [1, 2, 4, 8, 17], [34, 68, 136, 272, 544],
expected=578)
```

5.5 Finally, define a function named **multiplyEth** that takes in two arguments (the numbers that are going to be multiplied) and returns the result of multiplying the first number by the second number. You should use the **halveIt** and **doubleIt** functions inside this function. Then, pass the results returned from the two helper functions to the **sumLists** function.

```
#TODO: Write multiplyEth function
def multiplyEth(a, b):
    # Put your code below this comment
    return False

# TEST_CASE
# Note: if an error is given saying "check" is not defined, rerun the
```

```
first code
# cell in this notebook.

check(multiplyEth, 17, 34, expected=578)
check(multiplyEth, 15, 15, expected=225)
check(multiplyEth, 24, 34, expected=816)
```

Exercise 6: Factor []

6.1 Write a function `factor` that given a positive integer `n` and a positive integer `k`, finds the largest number `i` such that `n % (k**i) == 0`.

```
#TODO: Write factor function
```

Run the following code to check your solution.

```
check(factor, 100, 5, expected=2)
check(factor, 100, 2, expected=2)
check(factor, 64, 2, expected=6)
check(factor, 1, 2, expected=0)
```

6.2 Find the largest number `i` such that `10000! % (10 ** i) == 0`.

Hint: Use `math`

```
#TODO: Find largest i such that 10000! % (10 ** i) == 0
```

Exercise 7: Pascal Triangle []

The first seven layers of the Pascal Triangle looks like the following:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
```

Each layer of the Pascal Triangle can be computed from the previous one. Each number in one layer is the sum of the two numbers on top of it.

For example, the fourth layer

```
1 3 3 1
```

is computed in this way

$$1(1+2)(2+1)1$$

Define a function `nextLayer` which takes in a list `layer1` that represents one layer of the Pascal Triangle and returns a list of numbers representing the next layer.

For example, `nextLayer([1, 2, 1]) = [1, 3, 3, 1]`

```
# TODO: define nextLayer function
def nextLayer(layer1):
    pass #delete this line and add your code here

check(nextLayer,[1, 2, 1],expected=[1, 3, 3, 1])
check(nextLayer,[1],expected=[1, 1])
check(nextLayer,[1, 3, 3, 1],expected=[1, 4, 6, 4, 1])
```

7.2 Write a function `PascalGen` that takes in a positive integer `n`, and returns a (nested) list containing the first `n` layers in the Pascal Triangle.

```
# TODO: define PascalGen function
def PascalGen(n):
    pass #delete this line and add your code here

PascalGen(6)
```

7.3 Draw the first 5 layers of the Pascal triangle. Hint: If you want make the triangle looks like a pyramid, you can try to find out the length of the "base" of the pyramid.

```
# TODO: Draw first 5 layers of Pascal Triangle
```